

50+ Advanced SQL Interview Questions

1. Write a query to find the top 3 highest-paid employees in each department

LinkedIn@AshishZope

```
SELECT *
FROM (
    SELECT e.*, DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary
    DESC) AS salary_rank
    FROM employees e
) ranked
WHERE salary_rank <= 3;
```

2. Write a query to find the second-highest salary in each department, including departments with only one employee

LinkedIn@AshishZope

```
SELECT department_id, MAX(salary) FILTER (WHERE salary_rank = 2) AS
second_highest_salary
FROM (
    SELECT department_id, salary,
    DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC)
AS salary_rank
    FROM employees
) ranked
GROUP BY department_id;
```

3. Write a query to find employees whose salary is higher than the average salary of their department

LinkedIn@AshishZope

```
SELECT e.*
FROM employees e
JOIN (
    SELECT department_id, AVG(salary) AS department_average
    FROM employees GROUP BY department_id
) a USING (department_id)
WHERE e.salary > a.department_average;
```

4. Write a query to find employees whose salary is higher than their manager's salary

LinkedIn@AshishZope

```
SELECT e.employee_id, e.employee_name, e.salary, m.employee_name AS
manager_name, m.salary AS manager_salary
FROM employees e
JOIN employees m ON m.employee_id = e.manager_id
WHERE e.salary > m.salary;
```

5. Write a query to find the highest-paid employee in each department, returning all employees in case of a salary tie

LinkedIn@AshishZope

```
SELECT e.*
FROM employees e
JOIN (
    SELECT department_id, MAX(salary) AS max_salary FROM employees GROUP BY
    department_id
) d USING (department_id)
WHERE e.salary = d.max_salary;
```

6. Write a query to find the 3rd highest distinct salary company-wide without using **LIMIT, TOP, or OFFSET**

LinkedIn@AshishZope

```
SELECT salary
FROM (SELECT DISTINCT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS
salary_rank FROM employees) s
WHERE salary_rank = 3;
```

7. Write a query to find employees whose salary is in the top 10% of salaries within their department

LinkedIn@AshishZope

```
SELECT *
FROM (
    SELECT e.*, CUME_DIST() OVER (PARTITION BY department_id ORDER BY salary
DESC) AS top_fraction
    FROM employees e
) ranked
WHERE top_fraction <= 0.10;
```

8. Write a query to assign employees into salary buckets using **NTILE(4) within each department**

LinkedIn@AshishZope

```
SELECT e.*, NTILE(4) OVER (PARTITION BY department_id ORDER BY salary DESC) AS  
salary_bucket  
FROM employees e;
```

9. Write a query to find the employee whose salary is closest to the department average salary

LinkedIn@AshishZope

```
WITH averages AS (  
    SELECT e.*, AVG(salary) OVER (PARTITION BY department_id) AS  
    department_average  
    FROM employees e  
) , ranked AS (  
    SELECT *, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY ABS(salary  
- department_average), employee_id) AS rn  
    FROM averages  
)  
SELECT * FROM ranked WHERE rn = 1;
```

10. Write a query to find employees whose salary is greater than the salary of at least 3 other employees in their department

LinkedIn@AshishZope

```
SELECT e.*  
FROM employees e  
WHERE (SELECT COUNT(*) FROM employees x  
    WHERE x.department_id = e.department_id AND x.salary < e.salary) >= 3;
```

🔥 Deduplication / Latest Records

11. A customer can have multiple records. Write a query to keep only the latest record for each customer based on `updated_at`

LinkedIn@AshishZope

```
SELECT * FROM (
    SELECT cr.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
updated_at DESC, record_id DESC) AS rn
    FROM customer_records cr
) x WHERE rn = 1;
```

12. Find duplicate customer records based on: `customer_id + mobile_no` but keep the record with the latest `updated_at`

LinkedIn@AshishZope

```
SELECT * FROM (
    SELECT cr.*, ROW_NUMBER() OVER (PARTITION BY customer_id, mobile_no ORDER
BY updated_at DESC, record_id DESC) AS rn
    FROM customer_records cr
) x WHERE rn > 1;
```

13. Find customers whose email changed over time and display their previous and current email

LinkedIn@AshishZope

```
WITH changes AS (
    SELECT cr.*, LAG(email) OVER (PARTITION BY customer_id ORDER BY updated_at,
record_id) AS previous_email
    FROM customer_records cr
)
SELECT customer_id, previous_email, email AS current_email, updated_at
FROM changes WHERE previous_email IS NOT NULL AND email IS DISTINCT FROM
previous_email;
```

14. Given multiple status records per customer, return the latest status for every customer

LinkedIn@AshishZope

```
SELECT * FROM (
    SELECT csh.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
status_at DESC, status_id DESC) AS rn
    FROM customer_status_history csh
) x WHERE rn = 1;
```

15. Find customers whose latest status is ACTIVE but previous status was CLOSED

LinkedIn@AshishZope

```
WITH ranked AS (  
    SELECT csh.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY  
status_at DESC, status_id DESC) AS rn  
    FROM customer_status_history csh  
)  
SELECT customer_id  
FROM ranked  
WHERE rn <= 2  
GROUP BY customer_id  
HAVING COUNT(*) = 2  
    AND COUNT(*) FILTER (WHERE rn = 1 AND status = 'ACTIVE') = 1  
    AND COUNT(*) FILTER (WHERE rn = 2 AND status = 'CLOSED') = 1;
```

16. Find the first record and latest record for every customer in a single query

LinkedIn@AshishZope

```
SELECT customer_id,  
    ARRAY_AGG(record_id ORDER BY updated_at, record_id)[1] AS  
first_record_id,  
    ARRAY_AGG(record_id ORDER BY updated_at DESC, record_id DESC)[1] AS  
latest_record_id  
FROM customer_records GROUP BY customer_id;
```

17. Delete duplicate records while retaining the record having the minimum ID

LinkedIn@AshishZope

```
DELETE FROM customer_records duplicate  
USING customer_records keeper  
WHERE duplicate.customer_id = keeper.customer_id  
    AND duplicate.mobile_no IS NOT DISTINCT FROM keeper.mobile_no  
    AND duplicate.record_id > keeper.record_id;
```

18. Find customers having more than one record on the same date, but with different statuses

LinkedIn@AshishZope

```
SELECT customer_id, updated_at::date AS record_date  
FROM customer_records  
GROUP BY customer_id, updated_at::date  
HAVING COUNT(*) > 1 AND COUNT(DISTINCT status) > 1;
```

Complex JOIN Questions

19. Find employees who earn more than their manager

LinkedIn@AshishZope

```
SELECT e.employee_id, e.employee_name, e.salary, m.employee_name AS
manager_name
FROM employees e JOIN employees m ON m.employee_id = e.manager_id
WHERE e.salary > m.salary;
```

20. Find managers who manage more than 5 employees

LinkedIn@AshishZope

```
SELECT m.employee_id, m.employee_name, COUNT(e.employee_id) AS report_count
FROM employees m JOIN employees e ON e.manager_id = m.employee_id
GROUP BY m.employee_id, m.employee_name HAVING COUNT(*) > 5;
```

21. Find employees who have the same salary as another employee in the same department

LinkedIn@AshishZope

```
SELECT e.*
FROM employees e
WHERE EXISTS (
    SELECT 1 FROM employees x
    WHERE x.department_id = e.department_id AND x.salary = e.salary AND
    x.employee_id <> e.employee_id
);
```

22. Find customers who purchased Product A but never purchased Product B

LinkedIn@AshishZope

```
SELECT DISTINCT o.customer_id
FROM orders o JOIN order_items oi USING (order_id) JOIN products p USING
(product_id)
WHERE p.product_name = 'Product A'
AND NOT EXISTS (
    SELECT 1 FROM orders ob JOIN order_items oib USING (order_id) JOIN
products pb USING (product_id)
    WHERE ob.customer_id = o.customer_id AND pb.product_name = 'Product
B'
);
```

23. Find customers who purchased all products belonging to a particular category

LinkedIn@AshishZope

```

SELECT o.customer_id
FROM orders o JOIN order_items oi USING (order_id) JOIN products p USING
(product_id)
WHERE p.category_id = 10
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.product_id) = (SELECT COUNT(*) FROM products WHERE
category_id = 10);

```

24. Find products that were purchased by at least 80% of active customers

LinkedIn@AshishZope

```

WITH active AS (SELECT COUNT(*)::numeric AS total FROM customers WHERE status =
'ACTIVE'),
purchases AS (
    SELECT p.product_id, COUNT(DISTINCT o.customer_id)::numeric AS buyers
    FROM products p JOIN order_items oi USING (product_id) JOIN orders o USING
(order_id)
    JOIN customers c USING (customer_id) WHERE c.status = 'ACTIVE'
    GROUP BY p.product_id
)
SELECT p.product_id, p.product_name
FROM purchases x JOIN products p USING (product_id) CROSS JOIN active
WHERE x.buyers / NULLIF(active.total, 0) >= 0.80;

```

25. Find customers who placed orders in every month of 2025

LinkedIn@AshishZope

```

SELECT o.customer_id
FROM orders o
WHERE o.order_date >= DATE '2025-01-01' AND o.order_date < DATE '2026-01-01'
GROUP BY o.customer_id
HAVING COUNT(DISTINCT DATE_TRUNC('month', o.order_date)) = 12;

```

26. Find customers whose total purchase amount is greater than the average purchase amount of all customers

LinkedIn@AshishZope

```

WITH totals AS (SELECT customer_id, SUM(total_amount) AS total_spend FROM
orders GROUP BY customer_id)
SELECT * FROM totals WHERE total_spend > (SELECT AVG(total_spend) FROM totals);

```

27. Find departments where the average employee salary is higher than the company-wide average salary

LinkedIn@AshishZope

```
SELECT department_id, AVG(salary) AS department_average
FROM employees GROUP BY department_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);
```

🔥 Time-Series / Date Problems

28. Find users who logged in on 3 consecutive days

LinkedIn@AshishZope

```
WITH days AS (SELECT DISTINCT user_id, login_date::date AS login_date FROM
logins),
groups AS (SELECT *, login_date - (ROW_NUMBER() OVER (PARTITION BY user_id
ORDER BY login_date))::int AS grp FROM days)
SELECT user_id, MIN(login_date) AS streak_start, MAX(login_date) AS streak_end
FROM groups GROUP BY user_id, grp HAVING COUNT(*) >= 3;
```

29. Find users who logged in for 7 consecutive days

LinkedIn@AshishZope

```
WITH days AS (SELECT DISTINCT user_id, login_date::date AS login_date FROM
logins),
groups AS (SELECT *, login_date - (ROW_NUMBER() OVER (PARTITION BY user_id
ORDER BY login_date))::int AS grp FROM days)
SELECT DISTINCT user_id FROM groups GROUP BY user_id, grp HAVING COUNT(*) >= 7;
```

30. Find the longest consecutive login streak for every user

LinkedIn@AshishZope

```
WITH days AS (SELECT DISTINCT user_id, login_date::date AS login_date FROM
logins),
groups AS (SELECT *, login_date - (ROW_NUMBER() OVER (PARTITION BY user_id
ORDER BY login_date))::int AS grp
FROM days), streaks AS (SELECT user_id, grp, COUNT(*) AS
streak_length FROM groups GROUP BY user_id, grp)
SELECT user_id, MAX(streak_length) AS longest_streak FROM streaks GROUP BY
user_id;
```

31. Find customers who made purchases in 3 consecutive months

LinkedIn@AshishZope

```
WITH months AS (SELECT DISTINCT customer_id, DATE_TRUNC('month',
transaction_date)::date AS month FROM transactions),
groups AS (SELECT *, month - (ROW_NUMBER() OVER (PARTITION BY customer_id ORDER
BY month))::int * INTERVAL '1 month' AS grp FROM months)
SELECT customer_id, MIN(month) AS start_month, MAX(month) AS end_month
FROM groups GROUP BY customer_id, grp HAVING COUNT(*) >= 3;
```

32. Find the first purchase date and second purchase date for every customer and calculate the number of days between them

LinkedIn@AshishZope

```
WITH purchases AS (
SELECT customer_id, transaction_date::date AS purchase_date,
ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
transaction_date, transaction_id) AS rn
FROM transactions
)
SELECT customer_id, MIN(purchase_date) FILTER (WHERE rn = 1) AS
first_purchase_date,
MIN(purchase_date) FILTER (WHERE rn = 2) AS second_purchase_date,
MIN(purchase_date) FILTER (WHERE rn = 2) - MIN(purchase_date) FILTER
(WHERE rn = 1) AS days_between
FROM purchases GROUP BY customer_id;
```

33. Calculate month-over-month revenue growth percentage

LinkedIn@AshishZope

```
WITH monthly AS (SELECT DATE_TRUNC('month', order_date)::date AS month,
SUM(total_amount) AS revenue FROM orders GROUP BY 1)
SELECT month, revenue, LAG(revenue) OVER (ORDER BY month) AS previous_revenue,
ROUND(100 * (revenue - LAG(revenue) OVER (ORDER BY month)) /
NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 2) AS growth_pct
FROM monthly ORDER BY month;
```

34. Calculate year-over-year revenue growth percentage

LinkedIn@AshishZope

```
WITH yearly AS (SELECT EXTRACT(YEAR FROM order_date)::int AS year,
SUM(total_amount) AS revenue FROM orders GROUP BY 1)
SELECT year, revenue, LAG(revenue) OVER (ORDER BY year) AS previous_revenue,
ROUND(100 * (revenue - LAG(revenue) OVER (ORDER BY year)) /
NULLIF(LAG(revenue) OVER (ORDER BY year), 0), 2) AS growth_pct
FROM yearly ORDER BY year;
```

35. Find the 7-day rolling average revenue

LinkedIn@AshishZope

```
WITH daily AS (SELECT order_date::date AS sale_date, SUM(total_amount) AS
revenue FROM orders GROUP BY 1)
SELECT sale_date, revenue, AVG(revenue) OVER (ORDER BY sale_date RANGE BETWEEN
INTERVAL '6 days' PRECEDING AND CURRENT ROW) AS rolling_7_day_avg
FROM daily ORDER BY sale_date;
```

36. Find the 30-day rolling sum of transactions for every customer

LinkedIn@AshishZope

```
SELECT t.customer_id, t.transaction_date::date,
SUM(t.amount) OVER (PARTITION BY t.customer_id ORDER BY
t.transaction_date RANGE BETWEEN INTERVAL '29 days' PRECEDING AND CURRENT ROW)
AS rolling_30_day_sum
FROM transactions t;
```

37. Find customers who made their second purchase within 30 days of their first purchase

```
WITH purchases AS (  
    SELECT customer_id, transaction_date::date AS purchase_date,  
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY  
transaction_date, transaction_id) AS rn  
    FROM transactions  
)  
SELECT customer_id  
FROM purchases  
GROUP BY customer_id  
HAVING MAX(purchase_date) FILTER (WHERE rn = 2) <= MIN(purchase_date) FILTER  
(WHERE rn = 1) + 30;
```

🔥 Gaps & Islands

38. Given customer transactions, identify periods where the customer had no transaction for more than 30 days

```
WITH ordered AS (  
    SELECT customer_id, transaction_date::date AS transaction_date,  
           LEAD(transaction_date::date) OVER (PARTITION BY customer_id ORDER BY  
transaction_date) AS next_date  
    FROM transactions  
)  
SELECT customer_id, transaction_date AS period_start, next_date AS period_end,  
       next_date - transaction_date AS gap_days  
FROM ordered WHERE next_date - transaction_date > 30;
```

39. Given employee attendance data, find employees who were absent for 3 or more consecutive working days

LinkedIn@AshishZope

```

WITH absent AS (
    SELECT employee_id, attendance_date::date AS attendance_date,
           attendance_date::date - (ROW_NUMBER() OVER (PARTITION BY employee_id
ORDER BY attendance_date))::int AS grp
    FROM attendance WHERE status = 'ABSENT'
)
SELECT employee_id, MIN(attendance_date) AS absent_from, MAX(attendance_date)
AS absent_to, COUNT(*) AS absent_days
FROM absent GROUP BY employee_id, grp HAVING COUNT(*) >= 3;

```

40. Given daily sales data, identify continuous periods where sales were greater than ₹1 lakh

LinkedIn@AshishZope

```

WITH qualified AS (SELECT sale_date::date AS sale_date FROM daily_sales WHERE
total_sales > 100000),
groups AS (SELECT *, sale_date - (ROW_NUMBER() OVER (ORDER BY sale_date))::int
AS grp FROM qualified)
SELECT MIN(sale_date) AS period_start, MAX(sale_date) AS period_end, COUNT(*)
AS days
FROM groups GROUP BY grp;

```

41. Given user activity data, identify each user's continuous active period

LinkedIn@AshishZope

```

WITH active AS (
    SELECT user_id, activity_date::date AS activity_date,
           activity_date::date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER
BY activity_date))::int AS grp
    FROM user_activity WHERE is_active
)
SELECT user_id, MIN(activity_date) AS active_from, MAX(activity_date) AS
active_to, COUNT(*) AS active_days
FROM active GROUP BY user_id, grp;

```

42. Find the first and last transaction date for each customer and the number of active days

LinkedIn@AshishZope

```

SELECT customer_id, MIN(transaction_date)::date AS first_transaction_date,
       MAX(transaction_date)::date AS last_transaction_date,
       COUNT(DISTINCT transaction_date::date) AS active_days
FROM transactions GROUP BY customer_id;

```

Business Scenario SQL

43. Given an e-commerce order table, find the top 3 products by revenue for every month

LinkedIn@AshishZope

```
WITH monthly AS (  
    SELECT DATE_TRUNC('month', o.order_date)::date AS month, oi.product_id,  
           SUM(oi.quantity * oi.unit_price) AS revenue  
    FROM orders o JOIN order_items oi USING (order_id) GROUP BY 1, 2  
) , ranked AS (SELECT *, DENSE_RANK() OVER (PARTITION BY month ORDER BY revenue  
DESC) AS rn FROM monthly)  
SELECT * FROM ranked WHERE rn <= 3;
```

44. Find the top 2 customers by revenue in every month

LinkedIn@AshishZope

```
WITH monthly AS (  
    SELECT DATE_TRUNC('month', order_date)::date AS month, customer_id,  
    SUM(total_amount) AS revenue  
    FROM orders GROUP BY 1, 2  
) , ranked AS (SELECT *, DENSE_RANK() OVER (PARTITION BY month ORDER BY revenue  
DESC) AS rn FROM monthly)  
SELECT * FROM ranked WHERE rn <= 2;
```

45. Find customers whose monthly spending increased for 3 consecutive months

```

WITH monthly AS (
    SELECT customer_id, DATE_TRUNC('month', order_date)::date AS month,
    SUM(total_amount) AS spending
    FROM orders GROUP BY 1, 2
), changes AS (
    SELECT *, LAG(spending) OVER (PARTITION BY customer_id ORDER BY month) AS
previous_spending
    FROM monthly
), groups AS (
    SELECT *, month - (ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
month))::int * INTERVAL '1 month' AS grp
    FROM changes WHERE previous_spending IS NOT NULL AND spending >
previous_spending
)
SELECT customer_id, MIN(month) AS first_increase_month, MAX(month) AS
last_increase_month
FROM groups GROUP BY customer_id, grp HAVING COUNT(*) >= 3;

```

46. Find customers who were active last month but became inactive this month

```

WITH months AS (
    SELECT DATE_TRUNC('month', CURRENT_DATE)::date AS this_month,
    (DATE_TRUNC('month', CURRENT_DATE) - INTERVAL '1
month')::date AS last_month
), activity AS (
    SELECT DISTINCT customer_id, DATE_TRUNC('month', order_date)::date AS
month FROM orders
)
SELECT customer_id FROM activity a CROSS JOIN months m
WHERE a.month = m.last_month
    AND NOT EXISTS (SELECT 1 FROM activity b WHERE b.customer_id =
a.customer_id AND b.month = m.this_month);

```

47. Calculate customer retention percentage for Day-1, Day-7 and Day-30

```

WITH cohorts AS (
    SELECT customer_id, MIN(order_date::date) AS cohort_date FROM orders GROUP
    BY customer_id
), offsets AS (SELECT 1 AS day UNION ALL SELECT 7 UNION ALL SELECT 30)
SELECT o.day,
    ROUND(100.0 * COUNT(DISTINCT x.customer_id) / NULLIF(COUNT(DISTINCT
c.customer_id), 0), 2) AS retention_pct
FROM offsets o CROSS JOIN cohorts c
LEFT JOIN orders x ON x.customer_id = c.customer_id AND x.order_date::date =
c.cohort_date + o.day
GROUP BY o.day ORDER BY o.day;

```

48. Find the churn rate by month

```

WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date)::date AS month, customer_id
    FROM orders GROUP BY 1, 2
), periods AS (
    SELECT month, customer_id,
        LAG(month) OVER (PARTITION BY customer_id ORDER BY month) AS
previous_active_month
    FROM monthly
), counts AS (
    SELECT month, COUNT(*) FILTER (WHERE previous_active_month = month -
INTERVAL '1 month') AS churned_customers,
        COUNT(*) FILTER (WHERE previous_active_month IS NOT NULL) AS
previous_active_customers
    FROM periods GROUP BY month
)
SELECT month, churned_customers, previous_active_customers,
    ROUND(100.0 * churned_customers / NULLIF(previous_active_customers, 0),
2) AS churn_rate_pct
FROM counts ORDER BY month;

```

49. Find the percentage of total company revenue contributed by the top 10 customers.

```
WITH customer_revenue AS (  
    SELECT customer_id, SUM(total_amount) AS revenue FROM orders GROUP BY  
    customer_id  
) , top_customers AS (  
    SELECT customer_id, revenue, ROW_NUMBER() OVER (ORDER BY revenue DESC) AS  
    rn FROM customer_revenue  
)  
SELECT ROUND(100.0 * SUM(revenue) FILTER (WHERE rn <= 10) /  
NULLIF(SUM(revenue), 0), 2) AS top_10_revenue_pct  
FROM top_customers;
```

